

Хеш-функция

Материал из Википедии — свободной энциклопедии

Хеш-функция (англ. *hash function* от *hash* — «превращать в фарш», «мешанина»^[1]), или **функция свёртки** — функция, осуществляющая преобразование массива входных данных произвольной длины в выходную битовую строку установленной длины, выполняемое определённым алгоритмом. Преобразование, производимое хеш-функцией, называется **хешированием**. Исходные данные называются входным массивом, «**ключом**» или «**сообщением**». Результат преобразования называется «**хешем**», «**хеш-кодом**», «**хеш-суммой**», «**сводкой сообщения**».

Хеш-функции применяются в следующих случаях:

- при построении ассоциативных массивов;
- при поиске дубликатов в последовательностях наборов данных;
- при построении уникальных идентификаторов для наборов данных;
- при вычислении контрольных сумм от данных (сигнала) для последующего обнаружения в них ошибок (возникших случайно или внесённых намеренно), возникающих при хранении и/или передаче данных;
- при сохранении паролей *в системах защиты* в виде хеш-кода (для восстановления пароля по хеш-коду требуется функция, являющаяся обратной по отношению к использованной хеш-функции);
- при выработке электронной подписи (на практике часто подписывается не само сообщение, а его «хеш-образ»);
- и др.

В общем случае (согласно принципу Дирихле) нет однозначного соответствия между хеш-кодом и исходными данными. Возвращаемые хеш-функцией значения менее разнообразны, чем значения входного массива. Случай, при котором хеш-функция преобразует более чем один массив входных данных в одинаковые сводки, называется «**коллизией**». Вероятность возникновения коллизий используется для оценки качества хеш-функций.

Существует множество алгоритмов хеширования, различающихся различными свойствами. Примеры свойств:

- разрядность;
- вычислительная сложность;
- криптостойкость.

Выбор той или иной хеш-функции определяется спецификой решаемой задачи. Простейшим примером хеш-функции может служить «обрамление» данных циклическим избыточным кодом (англ. *CRC, cyclic redundancy code*).

Содержание

История

Виды «хеш-функций»

«Хеш-функции», основанные на делении

- 1. «Хеш-код» как остаток от деления на число всех возможных «хешей»
- 2. «Хеш-код» как набор коэффициентов получаемого полинома

«Хеш-функции», основанные на умножении

Хеширование строк переменной длины

Идеальное хеширование

Описание

Универсальное хеширование

Описание

Методы борьбы с коллизиями

Методы борьбы с коллизиями в хеш-таблицах

Криптографическая соль

Применение хеш-функций

Криптографические хеш-функции

Контрольные суммы

Геометрическое хеширование

Ускорение поиска данных

Примечания

Литература

Ссылки

История

Шифровка сообщений без возможности однозначной расшифровки, а только с целью подтверждения авторского приоритета, применялась издавна.

Галилео Галилей наблюдал кольца Сатурна, которые принял за «ушки». Не будучи уверен, но желая утвердить свой приоритет, он опубликовал сообщение с перестановленными буквами: *smaismrmilmepoetaleumibunenugttauiras*. В 1610 году он раскрыл исходную фразу: *Altissimum planetam tergeminum obseruau*, что в переводе с латинского языка означает «высочайшую планету тройною наблюдал». Таким образом, на момент публикации первого сообщения исходная фраза не была раскрыта, но была создана возможность подтвердить её позже.

В середине 1650-х Христиан Гюйгенс разглядел кольца и опубликовал сообщение с буквами, расставленными по алфавиту: *aaaaaaacccccdeeeeeghiiiiillmmnnnnnnnnnooooppqrrstttttuuuuu*. Через некоторое время была опубликована и исходная фраза: *Annulo cingitur, tenui plano, nusquam cohaerente, ad eclipticam inclinato* — «Окружен кольцом тонким, плоским, нигде не подвешенным, наклоненным к эклиптике». От применения хеш-функции, включая и цель позднее подтвердить некоторое нераскрытое сообщение, данный случай отличается только тем, что выходное сообщение не имеет фиксированной длины, а определяется длиной входного. Фактически, расстановка букв исходного сообщения по алфавиту является некоторой хеш-функцией, но только с результатом нефиксированной длины.

В январе 1953 года Ханс Петер Лун (нем. *Hans Peter Luhn*) (сотрудник фирмы IBM) предложил «хеш-кодирование». Дональд Кнут считает, что Ханс первым выдвинул систематическую идею «хеширования».

В 1956 году Арнольд Думи (англ. *Arnold Dumey*) в своей работе «*Computers and automation*» первым описал идею «хеширования» такой, какой её знает большинство программистов сейчас. Думи рассматривал «хеширование» как решение «проблемы словаря», предложил использовать в качестве «хеш-адреса» *остаток от деления на простое число*^[2].

В 1957 году в журнале «*IBM Journal of Research and Development*» была опубликована статья Уэсли Питерсона (англ. *W. Wesley Peterson*) о *поиске текста в больших файлах*. Эта работа считается первой «серьёзной» работой по «хешированию». В статье Уэсли определил «открытую адресацию», указал на уменьшение производительности при удалении. Спустя шесть лет была опубликована работа Вернера Бухгольца (нем. *Werner Buchholz*), в которой было проведено *обширное* исследование «хеш-функций». В течение нескольких последующих лет «хеширование» широко использовалось, однако никаких значимых работ не публиковалось.

В 1967 году «хеширование» в современном значении упомянуто в книге Херберта Хеллермана «*Принципы цифровых вычислительных систем*»^[3]. В 1968 году Роберт Моррис (англ. *Robert Morris*) опубликовал в журнале «*Communications of the ACM*» большой обзор по «хешированию». Эта работа считается «ключевой» публикацией, вводящей понятие о «хешировании» в *научный* оборот, и закрепившей термин «хеш», ранее применявшийся только специалистами (жаргон).

До начала 1990-х годов в русскоязычной литературе в качестве эквивалента термину «хеширование» благодаря работам Андрея Петровича Ершова использовалось слово «расстановка», а для «коллизий» использовался термин «конфликт» (А. П. Ершов использовал «расстановку» с 1956 года). В русскоязычном издании книги Никлауса Вирта «*Алгоритмы и структуры данных*» 1989 года также используется термин «расстановка». Предлагалось также назвать метод другим русским словом: «окрошка». Однако *ни один* из этих вариантов не прижился, и в русской литературе используется преимущественно термин «хеширование»^[4].

Виды «хеш-функций»

«Хорошая» хеш-функция должна удовлетворять двум **свойствам**:

- быстрое вычисление;
- минимальное количество «коллизий».

Введём обозначения:

- *K* — количество «ключей» (входных данных);
- *h(k)* — хеш-функция, имеющая не более *M* различных значений (выходных данных).

То есть:

$$\forall k \in [0; K) : h(k) < M.$$

В качестве примера «плохой» хеш-функции можно привести функцию с *M* = 1000, которая десятизначному натуральному числу *K* сопоставляет **три** цифры, выбранные из середины двадцатизначного квадрата числа *K*. Казалось бы, значения «хеш-кодов» должны *равномерно* распределяться между «000» и «999», но для «*реальных*» данных это справедливо лишь в том случае, если «*ключи*» не имеют «*большого*» количества нулей слева или справа^[4].

Рассмотрим несколько простых и надёжных реализаций «хеш-функций».

«Хеш-функции», основанные на делении

1. «Хеш-код» как остаток от деления на число всех возможных «хешей»

Хеш-функция может вычислять «хеш» как остаток от деления входных данных на M :

$$h(k) = k \bmod M,$$

где M — количество всех возможных «хешей» (выходных данных).

При этом очевидно, что при чётном M значение функции будет чётным при чётном k и нечётным — при нечётном k . Также не следует использовать в качестве M степень основания системы счисления компьютера, так как «хеш-код» будет зависеть только от нескольких цифр числа k , расположенных справа, что приведёт к большому количеству коллизий. На практике обычно выбирают простое M ; в большинстве случаев этот выбор вполне удовлетворителен.

2. «Хеш-код» как набор коэффициентов получаемого полинома

Хеш-функция может выполнять деление входных данных на полином по модулю два. В данном методе M должна являться степень двойки, а бинарные ключи ($K = k_{n-1}k_{n-2}\dots k_0$) представляются в виде полиномов, в качестве «хеш-кода» «берутся» значения коэффициентов полинома, полученного как остаток от деления входных данных K на заранее выбранный полином P степени m :

$$\begin{aligned} K(x) \bmod P(x) &= h_{m-1}x^{m-1} + \dots + h_1x + h_0 \\ h(x) &= h_{m-1}\dots h_1h_0 \end{aligned}$$

При правильном выборе $P(x)$ гарантируется отсутствие коллизий между почти одинаковыми ключами^[4].

«Хеш-функции», основанные на умножении

Обозначим символом w количество чисел, представимых машинным словом. Например, для 32-разрядных компьютеров, совместимых с IBM PC, $w = 2^{32}$.

Выберем некую константу A так, чтобы A была взаимно простой с w . Тогда хеш-функция, использующая умножение, может иметь следующий вид:

$$h(K) = \left[M \left\lfloor \frac{A}{w} * K \right\rfloor \right]$$

В этом случае на компьютере с двоичной системой счисления M является степенью двойки, и $h(K)$ будет состоять из старших битов правой половины произведения $A * K$.

Одним из преимуществ хеш-функций, основанных на делении и умножении, является выгодное использование неслучайности реальных ключей. Например, если ключи представляют собой арифметическую прогрессию (например, последовательность имён

«Имя 1», «Имя 2», «Имя 3»), хеш-функция, использующая умножение, отобразит арифметическую прогрессию в приближенно арифметическую прогрессию различных хеш-значений, что уменьшит количество коллизий по сравнению со случайной ситуацией^[4].

Одной из хеш-функций, использующих умножение, является хеш-функция, использующая хеширование Фибоначчи. Хеширование Фибоначчи основано на свойствах золотого сечения. В качестве константы *A* здесь выбирается целое число, ближайшее к $\varphi^{-1} * w$ и взаимно простое с *w*, где φ — это золотое сечение^[4].

Хеширование строк переменной длины

Вышеизложенные методы применимы и в том случае, если необходимо рассматривать ключи, состоящие из нескольких слов, или ключи переменной длины.

Например, можно скомбинировать слова в одно при помощи сложения по модулю *w* или операции «исключающее или». Одним из алгоритмов, работающих по такому принципу, является хеш-функция Пирсона.

Хеширование Пирсона — алгоритм, предложенный Питером Пирсоном (англ. *Peter Pearson*) для процессоров с 8-битовыми регистрами, задачей которого является быстрое преобразование строки произвольной длины в хеш-код. На вход функция получает слово *W*, состоящее из *n* символов, каждый размером 1 байт, и возвращает значение в диапазоне от 0 до 255. При этом значение хеш-кода зависит от каждого символа входного слова.

Алгоритм можно описать следующим псевдокодом, который получает на вход строку *W* и использует таблицу перестановок *T*:

```
h := 0
for each c in W loop
    index := h xor c
    h := T[index]
end loop
return h
```

Среди преимуществ алгоритма:

- простоту вычисления;
- отсутствие таких входных данных, для которых вероятность коллизии наибольшая;
- возможность модификации в идеальную хеш-функцию^[5].

В качестве альтернативного способа хеширования ключей *K*, состоящих из *l* символов ($K = x_1 x_2 \dots x_l$), можно предложить вычисление

$$h(K) = (h_1(x_1) + h_2(x_2) + \dots + h_l(x_l)) \mod M^{[4]}$$

Идеальное хеширование

Идеальной хеш-функцией (англ. *perfect hash function*) называется такая функция, которая отображает каждый ключ из набора S во множество целых чисел без коллизий. В математике такое преобразование называется инъективным отображением.

Описание

1. Функция $h(k): U \rightarrow [m]$ называется идеальной хеш-функцией для $S \subseteq U$, если она инъективна на S .
2. Функция $h(k): U \rightarrow [m]$ называется минимальной идеальной хеш-функцией для $S \subseteq U$, если она является идеальной хеш-функцией и $m = n = |S|$.
3. Для целого $k \geq 1$ функция $h(k): U \rightarrow [m]$ называется k -идеальной хеш-функцией (k -PHF) для $S \subseteq U$, если для каждого $j \in [m]$ имеем $|\{x \in S | h(x) = j\}| \leq k$.

Идеальное хеширование применяется, если требуется присвоить уникальный идентификатор ключу без сохранения какой-либо информации о ключе. Пример использования идеального (или скорее ***k***-идеального) хеширования: размещение хешей, связанных с данными, хранящимися в большой и медленной памяти, в небольшой и быстрой памяти. Размер блока можно выбрать таким, чтобы необходимые данные считывались из медленной памяти за один запрос. Подобный подход используется, например, в аппаратных маршрутизаторах. Также идеальное хеширование используется для ускорения работы алгоритмов на графах, если представление графа не уместится в основной памяти^[6].

Универсальное хеширование

Универсальным хешированием называется хеширование, при котором используется не одна конкретная хеш-функция, а происходит выбор хеш-функции из заданного семейства по случайному алгоритму. Универсальное хеширование обычно отличается низким числом коллизий, применяется, например, при реализации хеш-таблиц и в криптографии.

Описание

Предположим, что требуется отобразить ключи из пространства U в числа $[m]$. На входе алгоритм получает данные из некоторого набора $S \in U$ размерностью n . Набор заранее неизвестен. Как правило, алгоритм должен обеспечить наименьшее число коллизий, чего трудно добиться, используя какую-то определённую хеш-функцию. Число коллизий можно уменьшить, если каждый раз при хешировании выбирать хеш-функцию случайным образом. Хеш-функция выбирается из определённого набора хеш-функций, называемого универсальным семейством $H = \{h : U \rightarrow [m]\}$ ^[7].

Методы борьбы с коллизиями

Коллизией (иногда конфликтом^[2] или столкновением) называется случай, при котором одна хеш-функция для разных входных блоков возвращает одинаковые хеш-коды.

Методы борьбы с коллизиями в хеш-таблицах

Большинство первых работ, описывающих хеширование, посвящено методам борьбы с коллизиями в хеш-таблицах. Тогда хеш-функции применялись при поиске текста в файлах большого размера. Существует два основных метода борьбы с коллизиями в хеш-таблицах:

1. метод цепочек (метод прямого связывания);
2. метод открытой адресации.

При использовании метода цепочек в хеш-таблице хранятся пары «связный список ключей» — «хеш-код». Для каждого ключа хеш-функцией вычисляется хеш-код; если хеш-код был получен ранее (для другого ключа), ключ добавляется в существующий список ключей, парный хеш-коду; иначе создаётся новая пара «список ключей» — «хеш-код», и ключ добавляется в созданный список. В общем случае, если имеется N ключей и M списков, средний размер хеш-таблицы составит $\frac{N}{M}$. В этом случае при поиске по таблице по сравнению со случаем, в котором поиск выполняется последовательно, средний объём работ уменьшится примерно в M раз.

При использовании метода открытой адресации в хеш-таблице хранятся пары «ключ» — «хеш-код». Для каждого ключа хеш-функцией вычисляется хеш-код; пара «ключ» — «хеш-код» сохраняется в таблице. В этом случае при поиске по таблице по сравнению со случаем, в котором используются связные списки, ссылки не используются, выполняется последовательный перебор пар «ключ» — «хеш-код», перебор прекращается после обнаружения нужного ключа. Последовательность, в которой просматриваются ячейки таблицы, называется последовательностью проб^[4].

Криптографическая соль

Для защиты паролей и цифровых подписей от подделки создано несколько методов, работающих даже в том случае, если криптоаналитику известны способы построения коллизий для используемой хеш-функции. Одним из таких методов является добавление к входным данным так называемой криптографической «соли» — строки случайных данных; иногда «соль» добавляется и к хеш-коду. Добавление случайных данных значительно затрудняет анализ итоговых хеш-таблиц. Данный метод, к примеру, используется при сохранении паролей в UNIX-подобных операционных системах.

Применение хеш-функций

Хеш-функции широко используются в криптографии.

Хеш используется как ключ во многих структурах данных — хеш-таблицах, фильтрах Блума и декартовых деревьях.

Криптографические хеш-функции

Среди множества существующих хеш-функций принято выделять криптографически стойкие, применяемые в криптографии, так как на них накладываются дополнительные требования. Для того, чтобы хеш-функция H считалась криптографически стойкой, она должна удовлетворять трём основным требованиям, на которых основано большинство применений хеш-функций в криптографии:

- **необратимость**: для заданного значения хеш-функции m должно быть вычислительно неосуществимо найти блок данных X , для которого $H(X) = m$;
- **стойкость к коллизиям первого рода**: для заданного сообщения M должно быть вычислительно неосуществимо подобрать другое сообщение N , для которого $H(N) = H(M)$;

- стойкость к *коллизиям второго рода*: должно быть вычислительно неосуществимо подобрать пару сообщений (M, M') , имеющих одинаковый хеш.

Данные требования не являются независимыми:

- обратимая функция нестойка к коллизиям первого и второго рода;
- функция, нестойкая к коллизиям первого рода, нестойка к коллизиям второго рода; обратное неверно.

Не доказано существование необратимых хеш-функций, для которых вычисление какого-либо прообраза заданного значения хеш-функции теоретически невозможно. Обычно нахождение обратного значения является лишь вычислительно сложной задачей.

Атака «дней рождения» позволяет находить коллизии для хеш-функции с длиной значений n бит в среднем за примерно $2^{n/2}$ вычислений хеш-функции. Поэтому n -битовая хеш-функция считается криптостойкой, если вычислительная сложность нахождения коллизий для неё близка к $2^{n/2}$.

Криптографические хеш-функции должны иметь лавинный эффект — при малейшем изменении аргумента значение функции сильно изменяется. В частности, значение хеша не должно давать утечки информации даже об отдельных битах аргумента. Это требование является залогом криптостойкости алгоритмов хеширования, хеширующих пользовательский пароль для получения ключа^[8].

Хеширование часто используется в алгоритмах электронно-цифровой подписи, где шифруется не само сообщение, а его хеш-код, что уменьшает время вычисления, а также повышает криптостойкость. Также в большинстве случаев вместо паролей хранятся значения их хеш-кодов.

Контрольные суммы

Алгоритмы вычисления контрольных сумм — несложные, быстрые и легко реализуемые аппаратно алгоритмы, используемые для защиты данных от непреднамеренных искажений, в том числе — от ошибок аппаратуры. С точки зрения математики такие алгоритмы являются хеш-функциями, вычисляющими контрольный код. Контрольный код применяется для обнаружения ошибок, которые могут возникнуть при передаче и хранении информации.

Алгоритмы вычисления контрольных сумм по скорости вычисления в десятки и сотни раз быстрее, чем криптографические хеш-функции, и значительно проще в аппаратном исполнении.

Платой за столь высокую скорость является отсутствие криптостойкости — возможность легко «подогнать» сообщение под заранее известную контрольную сумму. Также обычно разрядность контрольных сумм (типичное число: 32 бита) ниже, чем разрядность криптографических хешей (типичные числа: 128, 160 и 256 бит), что означает возможность возникновения непреднамеренных коллизий.

Простейшим алгоритмом вычисления контрольной суммы является деление сообщения (входных данных) на 32- или 16-битовые слова с последующим суммированием слов. Такой алгоритм применяется, например, в протоколах TCP/IP.

Как правило, алгоритмы вычисления контрольных сумм должны обнаруживать типичные аппаратные ошибки, например, должны обнаруживать несколько подряд идущих ошибочных бит до заданной длины. Семейство алгоритмов так называемых *«циклических избыточных кодов»* удовлетворяет этим требованиям. К ним относится, например, алгоритм *CRC32*, применяемый в устройствах *Ethernet* и в формате сжатия данных *ZIP*.

Контрольная сумма, например, может быть передана по каналу связи вместе с основным текстом (данными). На приёмном конце контрольная сумма может быть рассчитана заново и может сравниваться с переданным значением. Если будет обнаружено расхождение, то при передаче возникли искажения, и можно запросить повторную передачу.

Пример применения хеширования в быту — подсчёт количества чемоданов, перевозимых в багаже. Для проверки сохранности чемоданов не требуется проверять сохранность каждого чемодана, достаточно посчитать количество чемоданов при погрузке и выгрузке. Совпадение чисел будет означать, что ни один чемодан не потерян. То есть, число чемоданов является хеш-кодом.

Данный метод можно дополнить для защиты передаваемой информации от фальсификации (метод MAC). В этом случае хеширование производится криптостойкой функцией над сообщением, объединённым с секретным ключом, известным только отправителю и получателю сообщения. Криптоаналитик, перехватив сообщение и значение хеш-функции, не сможет восстановить код, то есть не сможет подделать сообщение (см. *имитозащита*).

Геометрическое хеширование

Геометрическое хеширование (*англ. *geometric hashing**) — метод, широко применяемый в компьютерной графике и вычислительной геометрии для решения задач на плоскости или в трёхмерном пространстве, например для нахождения ближайших пар точек среди множества точек или для поиска одинаковых изображений. Хеш-функция в данном методе обычно получает на вход какое-либо метрическое пространство и разделяет его, создавая сетку из клеток. Хеш-таблица в данном случае является массивом с двумя или более индексами и называется «файлом сетки» (*англ. *grid file**). Геометрическое хеширование применяется в телекоммуникациях при работе с многомерными сигналами^[9].

Ускорение поиска данных

Хеш-таблицей называется структура данных, позволяющая хранить пары вида «ключ» — «хеш-код» и поддерживающая операции поиска, вставки и удаления элемента. Хеш-таблицы применяются с целью ускорения поиска, например, при записи текстовых полей в базе данных может рассчитываться их хеш-код, и данные могут помещаться в раздел, соответствующий этому хеш-коду. Тогда при поиске данных надо будет сначала вычислить хеш-код текста, и сразу станет известно, в каком разделе их надо искать. То есть искать надо будет не по всей базе, а только по одному её разделу, а это ускоряет поиск.

Бытовым аналогом хеширования в данном случае может служить размещение слов в словаре в алфавитном порядке. Первая буква слова является его хеш-кодом, и при поиске просматривается не весь словарь, а только слова, начинающиеся на нужную букву.

Примечания

- ↑ Вирт2, 2010, с. 256.
- ↑ Вирт, 1989.
- ↑ Herbert Hellerman. Digital Computer System Principles. N.Y.: McGraw-Hill, 1967, 424 pp.

4. Кнут, 2007.
5. Pearson, Peter K. (June 1990), "Fast Hashing of Variable-Length Text Strings" (<http://epaperpress.com/vbhash/download/p677-pearson.pdf>) (PDF), *Communications of the ACM*, **33** (6): 677, doi:10.1145/78973.78978 (<https://doi.org/10.1145%2F78973.78978>), Архивировано (<http://web.archive.org/web/20160509061327/http://epaperpress.com/vbhash/download/p677-pearson.pdf>) (PDF) из оригинала 9 мая 2016, Дата обращения: 21 февраля 2017
6. *Djamal Belazzougui, Fabiano C. Botelho, Martin Dietzfelbinger*. Hash, displace, and compress (<http://cmph.sourceforge.net/papers/esa09.pdf>) (неопр.). — Springer Berlin / Heidelberg, 2009. Архивировано (<https://web.archive.org/web/20110724073514/http://cmph.sourceforge.net/papers/esa09.pdf>) 24 июля 2011 года.
7. *Miltersen, Peter Bro* Universal Hashing (<http://www.daimi.au.dk/~bromille/Notes/un.pdf>) (англ.) (PDF). Архивировано (<https://www.webcitation.org/5hmOaVlSI?url=http://www.daimi.au.dk/~bromille/Notes/un.pdf>) 24 июня 2009 года.
8. Шнайер, 2002.
9. Wolfson, H.J. & Rigoutsos, I (1997). Geometric Hashing: An Overview. (<http://graphics.stanford.edu/courses/cs468-01-winter/papers/wr-ghao-97.pdf>) Архивная копия (<https://web.archive.org/web/20120313214029/http://graphics.stanford.edu/courses/cs468-01-winter/papers/wr-ghao-97.pdf>) от 13 марта 2012 на [Wayback Machine](#) IEEE Computational Science and Engineering, 4(4), 10-21.

Литература

- *Брюс Шнайер*. Прикладная криптография. Протоколы, алгоритмы, исходные тексты на языке Си. — М.: Триумф, 2002. — ISBN 5-89392-055-4.
- *Дональд Кнут*. Искусство программирования. Том 3. Сортировка и поиск = The Art of Computer Programming, vol.3. Sorting and Searching. — 2-е издание. — М.: «Вильямс», 2007. — С. 824. — ISBN 0-201-89685-0.
- *Никлаус Вирт*. Алгоритмы и структуры данных. — М.: «Мир», 1989. — ISBN 5-03-001045-9.
- *Никлаус Вирт*. Алгоритмы и структуры данных. Новая версия для Оберона. — М.: «ДМК Пресс», 2010. — ISBN 978-5-94074-584-6.

Ссылки

- [Информация по электронной цифровой подписи \(http://protect.htmlweb.ru/ecp.htm\)](http://protect.htmlweb.ru/ecp.htm)
-

Источник — <https://ru.wikipedia.org/w/index.php?title=Хеш-функция&oldid=135090746>

Эта страница в последний раз была отредактирована 24 декабря 2023 в 09:50.

Текст доступен по лицензии Creative Commons «С указанием авторства — С сохранением условий» (CC BY-SA); в отдельных случаях могут действовать дополнительные условия.

Wikipedia® — зарегистрированный товарный знак некоммерческой организации Фонд Викимедиа (Wikimedia Foundation, Inc.)